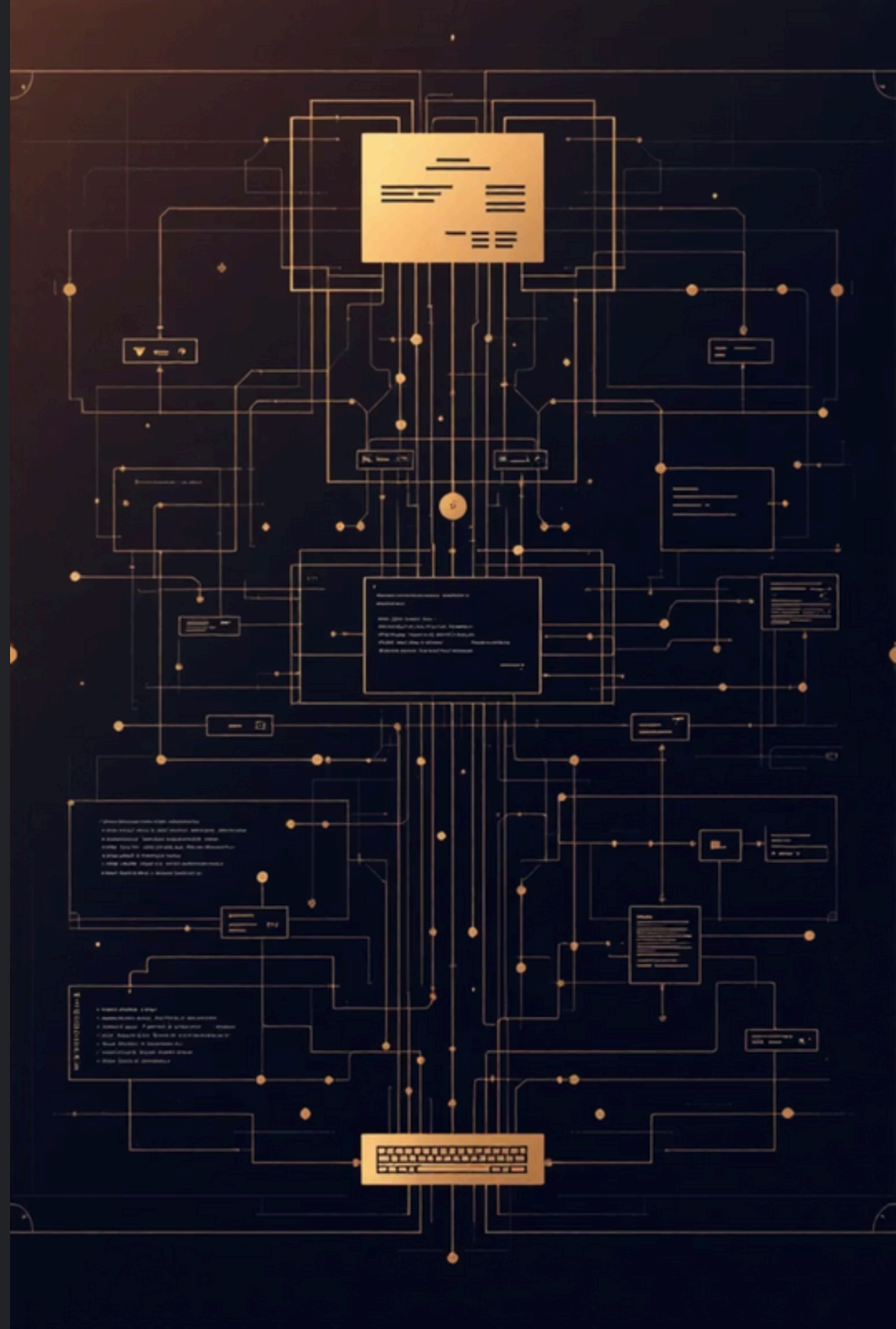
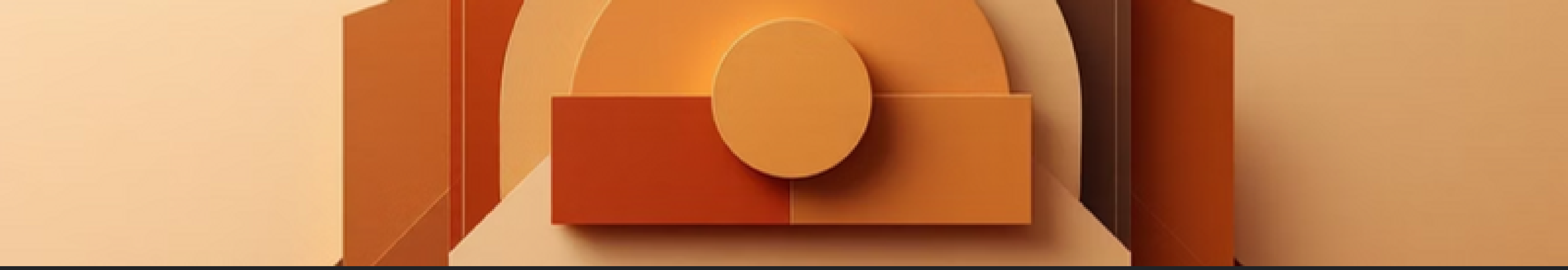
2. HAFTA: Yazılım Türleri ve Örnekleri

- Yazılım Türleri
- Yazılım Mühendisliği Hakkında Yanılgılar





Yazılım Türleri

Farklı alanlarda farklı görevler üstlendiği için yazılımlar, işlevlerine ve kullanım amaçlarına göre çeşitli türlere ayrılmıştır. Bu ayrım; donanımı yöneten sistem yazılımlarından, mühendislik ve bilimsel hesaplamalara odaklanan mühendislik yazılımlarına, cihazların içinde çalışan gömülü yazılımlardan, günlük hayatımızda kullandığımız uygulama yazılımlarına kadar geniş bir yelpazeyi kapsar.

Her yazılım türü, kendine özgü özelliklere, kullanım senaryolarına ve teknik gereksinimlere sahiptir. Bu çeşitlilik, modern teknoloji dünyasının karmaşıklığını ve zenginliğini yansıtır.

1. Sistem Yazılımı



Temel Altyapı



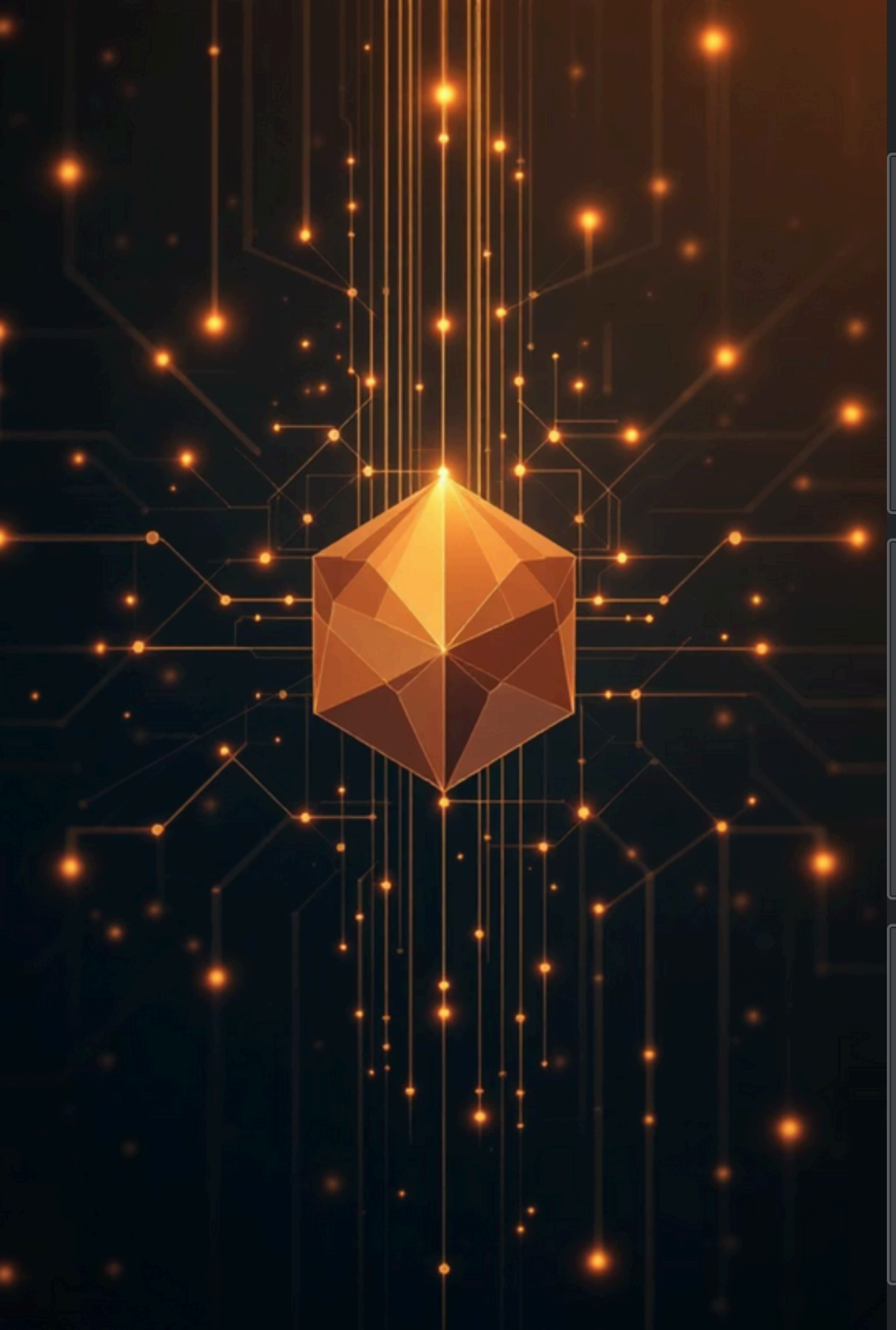
Kaynak Yönetimi



Temel Katman

Sistem yazılımı, bilgisayar donanımını yöneten ve diğer yazılımların (uygulamaların) sorunsuz çalışmasını sağlayan temel altyapı programlarını kapsar. Bu yazılımlar, donanım kaynaklarını (CPU, bellek, depolama gibi) verimli bir şekilde koordine eder, diğer programlara hizmet sunar ve sistemin genel işleyişini sağlar.

Derleyiciler (kaynak kodu makine diline çeviren araçlar), işletim sistemleri, sürücüler (device drivers) ve yardımcı araçlar gibi unsurları içerir. Temel amacı, kullanıcıya doğrudan hizmet etmek yerine, diğer yazılımların üzerinde bir "temel katman" oluşturmaktır. Bu tür yazılımlar genellikle donanım bağımlıdır ve sistemin stabilitesini sağlar.



Sistem Yazılımı Örnekleri

1

İşletim Sistemleri

Windows 11 veya Linux (Ubuntu), donanım kaynaklarını yönetir, dosya sistemini organize eder ve uygulamaların çalışmasını sağlar.

Örneğin, bir oyun oynarken arka planda bellek yönetimi yapan kısım sistem yazılımıdır. İşletim sistemi, her uygulamaya ne kadar bellek ayrılacağını belirler ve çakışmaları önler.

2

Derleyiciler

GCC (GNU Compiler Collection), C++ kodunu makine diline dönüştürür.

Bir geliştirici, yeni bir uygulama yazarken bu derleyiciyi kullanarak kodunu çalıştırılabilir hale getirir. Derleyici, insan tarafından okunabilir kodu, bilgisayarın anlayabileceği binary formata çevirir.

3

Sürücüler

Grafik kartı sürücüsü (NVIDIA CUDA), donanımın bilgisayar ile iletişimini sağlar.

Örneğin, bir video düzenleme yazılımı çalıştırırken ekran kartının tam performansını kullanmasını bu sürücü sağlar. Sürücüler olmadan, güçlü donanım bile kullanılamaz halde kalır.

2. Mühendislik ve Bilimsel Yazılım

Bu tür yazılımlar, mühendislik ve bilimsel alanlarda karmaşık hesaplamalar yapmak üzere tasarlanır. Genellikle büyük hacimli verilerle (big data) çalışır ve "number crunching" (rakam öğütme) olarak bilinen yoğun matematiksel işlemleri gerçekleştirir.

Temel Özellikler

- Simülasyonlar ve modellemeler
- Veri analizi ve optimizasyon
- Özel algoritmalar ve yüksek hassasiyet
- Hız ve performans odaklı

Kullanım Alanları

- Deneysel veri işleme
- Öngörü üretme
- Yüksek performanslı hesaplama (HPC)
- Bilimsel araştırma desteği



Bilim insanları ve mühendisler için özel algoritmalar içerir; hassasiyet ve hız ön plandadır. Bu yazılımlar, deneysel verileri işleyerek öngörüler üretir ve genellikle yüksek performanslı hesaplama ortamlarında kullanılır.

Mühendislik Yazılımı Örnekleri



MATLAB

Mühendislik simülasyonları için kullanılır. Örneğin, bir otomobil mühendisi, araç şasisinin titreşimini modellemek için büyük veri setlerini işleyerek stres analizleri yapar. MATLAB, matris hesaplamaları ve signal processing için optimize edilmiştir.



ANSYS

Bilimsel hesaplamalarda sonlu elemanlar analizi (FEA) için idealdir. Bir havacılık mühendisi, uçak kanadının aerodinamik yüklerini simüle ederek "rakam öğütme" yoluyla binlerce veri noktasını hesaplar. Her bir mesh elementi ayrı ayrı analiz edilir.



Python (NumPy/SciPy)

Bir astrofizikçi, galaksi verilerini analiz etmek için milyonlarca sayısal veriyi işler; örneğin, Hubble teleskobundan gelen görüntüleri dönüştürerek kara delik yörüngelerini hesaplar. Python'un esnekliği ve güçlü kütüphaneleri, bilimsel araştırmalarda vazgeçilmezdir.

3. Gömülü (Embedded) Yazılım

Gömülü yazılımlar, belirli bir donanım cihazına entegre edilmiş ve o donanımın doğrudan kontrolü için geliştirilmiş programlardır. Donanım ile çok sıkı bir ilişki içindedir; genellikle bellek ve işlem gücü sınırlıdır.

Denetim Amaçlı

Fiziksel sistemlerin kontrolü ve yönetimi için tasarlanmıştır

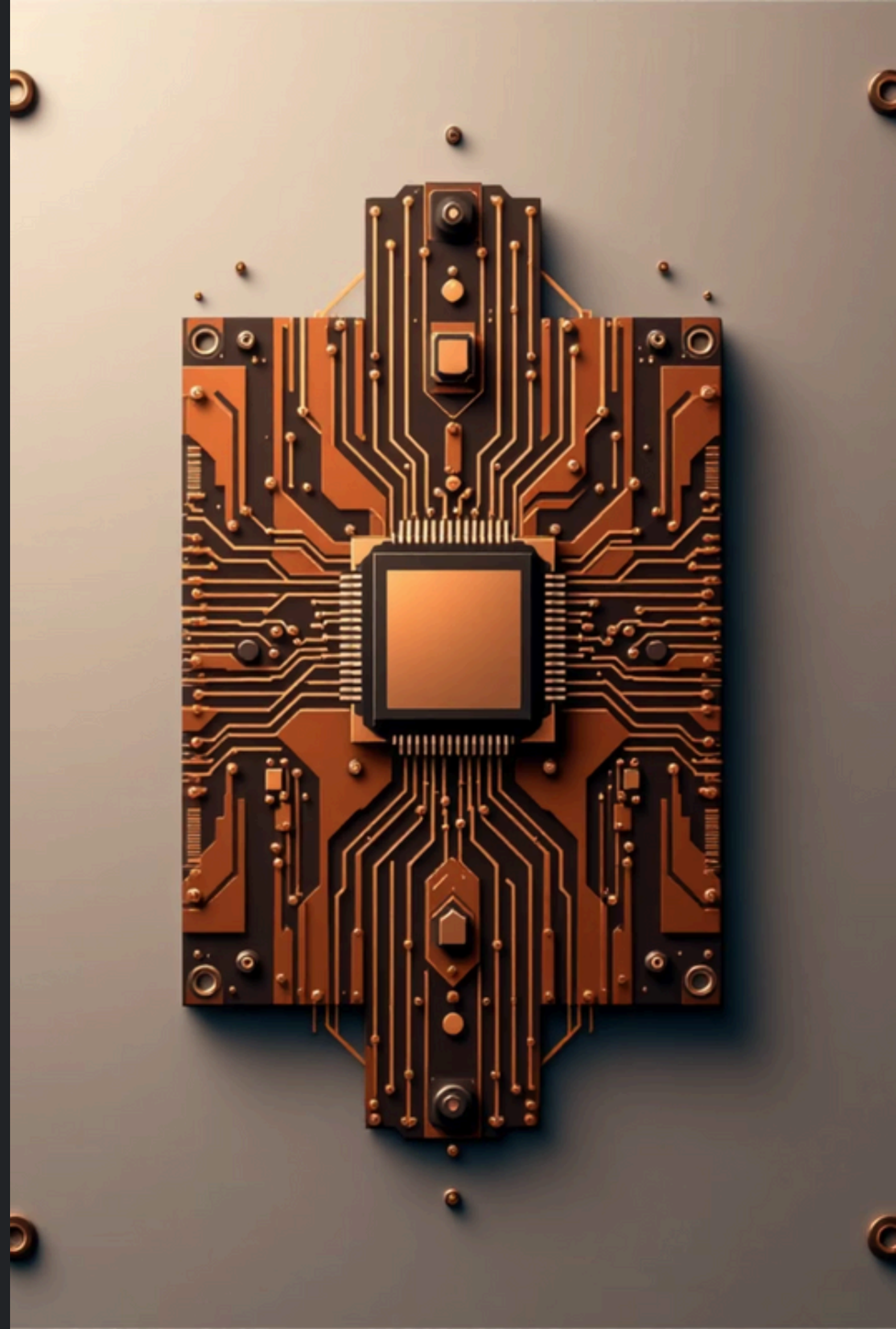
Gerçek Zamanlı

Milisaniyeler içinde tepki vermelidir, gecikme toleransı çok düşüktür

Düşük Hata Payı

Fiziksel sistemleri etkilediği için güvenilirlik kritik önemdedir

Bu yazılımlar, cihazın temel işlevlerini yönetir ve genellikle firmware olarak adlandırılır. Hata payı düşük olmalı, çünkü fiziksel sistemleri etkiler. Gömülü sistemler, ev aletlerinden otomobillere, medikal cihazlardan havacılık sistemlerine kadar her yerde bulunur.



Gömülü Yazılım Örnekleri



Tıbbi Cihazlar (Pacemaker)

Kalp pili içindeki gömülü yazılım, kalp ritmini donanım elektrotları ile denetler. Gerçek zamanlı olarak anormal ritimleri algılayıp elektrik impulsu gönderir, gecikme hayati risk yaratabilir.

Bu cihazlar, 7/24 kesintisiz çalışmalı ve son derece güvenilir olmalıdır. FDA gibi düzenleyici kurumlar tarafından sıkı testlerden geçirilir ve yazılım güncellemeleri bile hassas protokoller gerektirir.

Akıllı Saatler (Smartwatch)

Akıllı saatler gibi cihazlardaki kalp atış sensörü yazılımı, donanım sensörünü sürekli izler ve gerçek zamanlı veri üretir; örneğin, egzersiz sırasında nabzı milisaniyede hesaplayarak uyarı verir.

Optik sensörler ve gömülü algoritmalar, kan akışındaki değişimleri algılayarak kalp atış hızını hesaplar. Batarya tasarrufu için optimize edilmiş bu yazılımlar, günlerce çalışabilir.



---

4. Uygulama Yazılımı

Uygulama yazılımları, genel kullanıcı ihtiyaçlarını karşılamak üzere geliştirilmiş, hazır ve ticari olarak satılabilir programlardır. "Product-line" (ürün serisi), "shrink-wrapped" (kutu içinde hazır) veya "commercial off-the-shelf (COTS)" olarak bilinir.

Standartlara Uygunluk

Yazılım Mühendisliği'nde TS/BS ISO/IEC 25051 gibi standartlar, bu yazılımların kalitesini, uyumluluğunu ve yeniden kullanılabilirliğini tanımlar

Geniş Kullanıcı Kitlesi

Farklı müşteriler tarafından geniş ölçekte kullanılabilir; özelleştirme minimumdur ve kullanıcı dostu arayüzlere sahiptir



Otomasyon ve Verimlilik

Günlük işleri otomatikleştirir ve kullanıcının teknik bilgi gerektirmeden görevlerini tamamlamasını sağlar

Bu yazılımlar, ofis uygulamalarından muhasebe programlarına, grafik tasarım araçlarından proje yönetim sistemlerine kadar geniş bir yelpazede bulunur. Hepsinin ortak özelliği, son kullanıcıya değer katmak ve belirli işlevleri yerine getirmektir.

Uygulama Yazılımı Örnekleri



Kelime İşlem Uygulamaları

Microsoft Word, belgeleri düzenlemek için kullanılır. Bir öğrenci, makalesini yazarken metin formatlama, tablo ekleme gibi genel amaçlı özellikleri kullanır; kutudan hazır halde gelir ve milyonlarca kullanıcıya hitap eder.

Word, işbirliği özellikleri, şablon desteği ve çoklu format uyumluluğu ile profesyonel belge hazırlamayı kolaylaştırır. Aynı yazılım, öğrenciden profesyonele herkesin ihtiyacını karşılar.



Cari Hesap Uygulamaları

QuickBooks, küçük işletmeler için muhasebe otomasyonu sağlar. Bir mağaza sahibi, satışları kaydederek raporlar üretir; COTS standardına uyar ve farklı sektörlerde (perakende, hizmet) kullanılabilir.

QuickBooks, fatura oluşturma, stok takibi, vergi hesaplamaları ve finansal raporlama gibi işlevleri tek bir platformda toplar. ISO standartlarına göre test edilmiş, güvenilir bir çözümdür.



Çeşitli Otomasyon Programları

Adobe Photoshop, grafik tasarım otomasyonu için idealdir. Bir reklamcı, fotoğrafları filtreleyerek düzenler; ürün serisi olarak satılır ve ISO standartlarına göre test edilmiştir.

Photoshop, katmanlar, maskeler, filtreler ve yapay zeka destekli araçlarla yaratıcı süreci hızlandırır. Profesyonel fotoğrafçılardan sosyal medya yaratıcılarına kadar geniş bir kitle tarafından kullanılır.

5. Kurumsal Yazılım

Kurumsal yazılımlar, belirli bir şirketin veya organizasyonun ticari iş süreçlerine özel olarak tasarlanmış programlardır. İş akışlarını (business processes) optimize eder, veri dönüştürme, değerlendirme ve gerçek zamanlı izleme gibi işlevleri içerir.

Özelleştirme

Genellikle müşteriye özel (custom-made) geliştirilir, bu yüzden esneklik ve entegrasyon ön plandadır.

Adaptasyon

Zamanla değişen iş ihtiyaçlarına uyum sağlar, ancak uzun vadede "eski yazılım" (legacy) haline dönüşebilir.

Verimlilik

Bu yazılımlar, ERP (Enterprise Resource Planning) gibi sistemlerle ilişkilidir ve organizasyonel verimliliği artırır.



Kurumsal Yazılım'a Örnekler

- ERP Sistemleri (SAP): Bir üretim şirketi için özel tasarlanmış; stok yönetimi, sipariş işleme ve finansal raporlama gibi iş süreçlerini gerçek zamanlı izler. Örneğin, bir fabrika, hammaddelerin girişini otomatik dönüştürerek envanteri günceller.
- CRM Uygulamaları (Salesforce): Müşteri ilişkilerini yönetmek için özelleştirilir. Bir banka, müşteri verilerini değerlendirerek satış fırsatlarını gerçek zamanlı takip eder; iş süreçlerine göre modülleri uyarlanır.
- Veri Dönüştürme Uygulamaları: Bir lojistik firması için geliştirilmiş bir yazılım, sevkiyat verilerini XML'den JSON'a dönüştürerek tedarik zincirini izler. Zamanla, şirket büyüdükçe bu yazılım eski ihtiyaçlara uymaz ve legacy olur.



Eski Yazılım (Legacy Software)

Eski yazılım, bir organizasyonun iş süreçlerinin kritik bir parçası haline gelmiş, ancak çok uzun süredir (genellikle 10-20 yıl veya daha fazla) kullanılan ve güncelliğini yitirmiş programlardır.

> **Başlangıçta etkili olsa da, zamanla bakım maliyeti artar ve modern standartlara uymaz. Kurumsal yazılımların sıkça karşılaştığı bir durumdur; iş süreçlerini desteklerken, değişen teknolojiye ayak uyduramaz. Bu yazılımlar, sistemin "kalbi" gibi çalışır ama yeniliklere kapalıdır.**

Eski Yazılımın Dezavantajları

Eksik veya Hatalı Dokümantasyon

Kodun nasıl çalıştığına dair belgeler yetersiz veya kayıptır. Örnek: 1990'larda geliştirilmiş bir banka muhasebe sistemi, güncel geliştiricilerin anlaması için sadece eski notlar içerir; bu, hata ayıklamayı saatler yerine günler sürdürür.

Zamanla Karmaşıklaşmış Kod

Yıllar içinde eklemelerle "spagetti kod" haline gelir. Örnek: Bir hastane yönetim sistemi, yeni modüller eklendikçe dallanır; bir güncelleme, beklenmedik hatalara yol açar.

Esnek Olmayan Yapı

Değişiklik yapmak zordur, modüler değildir. Örnek: Eski bir envanter yazılımı, bulut entegrasyonu için yeniden yazılmak zorunda kalır çünkü monolitik bir yapıdadır.

Eski Donanımla Çok Sıkı İlişki

Modern donanımlarla uyumsuzdur. Örnek: COBOL tabanlı bir sigorta sistemi, 1980'lerin mainframe'lerine bağımlıdır; yeni sunuculara taşınmak pahalıdır.

Yazılım Mühendisliğindeki Gelişmelerden Yoksunluk Nedeniyle Düşük Kalite

Agile, CI/CD gibi modern pratiklerden habersizdir. Örnek: Güvenlik açıkları yamalanmaz, çünkü kod eski dilde (örneğin Fortran) yazılmıştır ve test araçları yetersizdir.



Eski yazılım neden değişmeli?

İş Alanındaki Yeni Gereksinimler

Pazar değişir, yazılım uymaz. Örnek: Bir perakende zincirinin legacy POS (satış noktası) sistemi, e-ticaret entegrasyonu için yetersiz kalır; mobil ödemeleri desteklemez.

Güncel Sistemlerle Birlikte Çalışabilmesi İçin Uyumluluk Kazandırılması

API'ler ve bulut servisleri ile entegre olamaz. Örnek: Eski bir CRM, modern AI tabanlı analitik araçlarla (örneğin Google Analytics) bağlanamaz; veri siloları oluşur.

Donanımın Ömrünün Dolması Nedeniyle Daha Güncel Ortama Taşınma Gerekliliği

Eski sunucular arızalanır. Örnek: Bir hükümet veritabanı, 20 yıllık sunuculardan AWS bulutuna taşınmak zorunda kalır; aksi takdirde veri kaybı riski artar.

Bu tür yazılımları yönetmek için "modernizasyon" stratejileri (refactoring, migration) kullanılır.

ÖDEV

SİZLERDE BU YAZILIM TÜRLERİNİN HER BİRİNE 5 FARKLI ÖRNEK BULUNUZ.



Yazılım Geliştirme Süreci Hakkında Yanılgılar

Yazılım geliştirme, karmaşık bir süreçtir ve bu alanda çeşitli yanılgılar, projelerin başarısızlığına yol açabilir. Yanılgılar, genellikle yazılımın uzun vadeli doğasını göz ardı etmekten kaynaklanır.



Programcı Açısından Yazılım Hakkındaki Yanılgılar

Programcılar, teknik odak nedeniyle bazı süreçleri hafife alabilir. Bu yanılgılar, bakım ve kalite sorunlarına yol açar.

1.Yanılgı: "Yazılımı tamamlayıp müşteriye teslim edince işimiz biter."

Bu, yazılım geliştirme sürecini "yaz ve unut" gibi gören bir yaklaşımdır. Gerçekte, yazılımın hayat döngüsünün (SDLC - Software Development Life Cycle) büyük kısmı teslimattan sonradır: Bakım, güncelleme, hata düzeltme ve uyumluluk çalışmaları toplam çabanın %60-80'ini oluşturur. Teslimat sadece bir kilometre taşıdır; yazılım, kullanıldıkça evrilir.

Örnek: Bir mobil bankacılık app'i geliştiren programcı, kodu teslim ettikten sonra yeni iOS güncellemeleriyle çökmeleri görmezden gelirse, kullanıcı kaybı yaşanır. Gerçekte, haftalık hotfix'ler ve kullanıcı geri bildirimleriyle 1-2 yıl bakım yapılır.

2.Yanılgı: "Yazılımı tamamlamadan kalitesini ölçemem."

Kalite, sadece son ürünle ölçülemez; erken aşamalarda (tasarım, kodlama) metrikler (kod kapsama oranı, cyclomatic complexity) ve incelemelerle (code review) uygulanabilir. Bu, hataları erken yakalar ve maliyeti %10'a indirir. Çözümleme aşamasında bile, gereksinimlerin netliği gibi ölçütler kullanılır.

Örnek: Bir web sitesi geliştirirken, kod yazmadan önce prototip testleri (usability testing) yapılırsa, UI hataları %50 azalır. Aksi takdirde, tam ürün hazır olunca keşfedilen bir hata, tüm sistemi yeniden yazdırmaya yol açar.

3.Yanılgı: "Yazılım eşittir program."

Yazılım sadece kod değildir; gereksinim analizi (requirements engineering), dokümantasyon, testler ve entegrasyon gibi aşamalar ayrı emek ister. Program, yazılımın sadece bir parçasıdır; diğerleri, sürdürülebilirlik sağlar. Toplam çaba, kodlamadan %40-50 daha fazladır.

Örnek: Bir ERP sistemi geliştirirken, kod %30 emek alırken, kullanıcı gereksinimlerini toplamak %20, entegrasyon testleri %25 harcar. Kod odaklı yaklaşırsanız, sistem çalışır ama kullanıcılar anlamaz veya veri kaybı olur.

4.Yanılgı: "Yazılım mühendisliğinin gereklerini uygulayarak boşuna çaba harcıyoruz."

Yazılım mühendisliği standartları (planlama, dokümantasyon, test) "gereksiz bürokrasi" gibi görünse de, harita gibi yol gösterir. Küresel istatistiklere göre (Standish Group raporları), standartlara uymayan projelerin %50'si başarısız olur. Erken yatırım, sonraki aşamalarda tasarruf sağlar (örneğin, hata düzeltme maliyeti 100 kat azalır).

Örnek: Bir e-ticaret platformunda Agile metodlarını atlayıp doğrudan kodlarsanız, özellik eklemek aylar alır. Standartlarla, sprint planlaması sayesinde proje %30 erken biter ve kalite yükselir.

Müşteri Açısından Yazılım Hakkındaki Yanılgılar

Müşteriler, yazılımı "sipariş ver ve al" gibi görür, ama bu belirsizliği artırır.

1 Yanılgı: "Programın yazılmasına başlanması için amaçları genel olarak belirlemek yeter, ayrıntılar sonra kararlaştırılabilir."

Belirsiz gereksinimler, temelsiz bir bina gibi çöker; erken detaylandırma (user stories, use cases) şarttır. Sonradan eklemeler, scope creep'e (kapsam kayması) yol açar ve bütçeyi %200 şişirir.

Örnek: Bir restoran zinciri, "sipariş app'i" derse geliştirici yanlış özellik ekler (örneğin, sadece online ödeme). Detaylı gereksinimle (menü entegrasyonu, offline mod), app ilk seferde uyumlu olur ve revizyon maliyeti düşer.

2 Yanılgı: "Yazılım esnektir. Değişen gereksinimler kolayca sisteme uyarlanabilir."

Yazılım, oyun hamuru değil, kil veya cam gibidir; erken değişimler ucuzdur, ama döngü ilerledikçe (tasarım > kod > test) maliyet üstel artar (Boehm's curve: 1 birim erken = 100 birim geç). Çevik (Agile) süreçler esnekliği artırır, ama yine de planlama şart.

Örnek: Bir CRM yazılımında, tasarım aşamasında "mobil uyum" eklemek 1 hafta alır; kodlandıktan sonra 3 ay. Bir şirket, piyasa değişimiyle özellik isterse, bütçeyi aşar ve proje gecikir.

İdari Açıdan Yazılım Hakkındaki Yanılgılar

Yöneticiler, kaynakları kısa vadeli yönetir, ama bu kaos yaratır.

01

Yanılgı: "İşler yetişmiyorsa takıma yeni programcılar ekleriz."

Brooks'un Kanunu'na göre (The Mythical Man-Month), ek ekip iletişim maliyetini artırır; yeni üyeler öğrenme eğrisiyle (ramp-up) gecikmeyi üstel büyütür. Erken planlama ve paralel çalışma daha iyidir.

Örnek: Bir proje 6 ay gerideyken 5 programcı eklenirse, eğitim ve koordinasyonla 9 aya uzar. Bunun yerine, görevleri modüllere ayırıp deneyimli ekiple, 3 ayda bitirilir.

02

Yanılgı: "Açık kaynak yazılım üretirsem kar edemem."

Açık kaynak (OSS), doğrudan satıştan kar etmese de, danışmanlık, destek hizmetleri, dual-licensing veya freemium modellerle gelir yaratır. Topluluk katkısı, marka değerini artırır (örneğin, Red Hat'in milyarlarca doları).

Örnek: WordPress açık kaynaklıdır, ama Automattic firması hosting ve eklenti danışmanlığıyla yıllık 300 milyon dolar kazanır. Bir startup, OSS araç geliştirip premium destek satarak pazara girer.